SonarQube with Jenkins

SonarQube

SonarQube is a code quality analysis tool it automatically analyzes source code to detect,

- **Bugs (**A bug is something that can cause your code to behave incorrectly)

```
def divide(a, b):
    return a / b  # ❌ Risk of ZeroDivisionError
```

- **Code smells** (These aren't bugs but poor practices that make code harder to read, maintain, or extend.)

```
public void processOrder(Order order) {
    // 50+ lines of code inside one method
}
// ❌ Large method: hard to test, read, or reuse (code smell)
```

- **Duplicated code**

- **Vulnerabilities** (These are issues that could be exploited by an attacker.)

```
String password = "myPassword123";
// ❌ Hardcoded credentials – a common security issue
```

- **Complex code structures**

```
if (a > b) {
    if (c > d) {
        if (e < f) {
            // ❌ Deeply nested logic – difficult to read and test
        }
    }
}
```

```
if (condition1 && condition2 || condition3 && !condition4 || (condition5 && condition6)) {
    // ❌ Complex boolean logic – needs simplification or comments
}
```

Jenkins

Jenkins is an open-source automation server used primarily for Continuous Integration (CI) and Continuous Delivery (CD) in software development.

Requirements

- JDK 17 or 21 (https://download.oracle.com/java/21/latest/jdk-21_windows-x64_bin.exe (sha256))
- Jenkins (https://www.jenkins.io/download/)
- SonarQube (SonarQube Download Success | Community Build | Sonar)
- Apache Maven (https://dlcdn.apache.org/maven/maven-3/3.9.11/binaries/apache-maven-3.9.11-bin.zip)

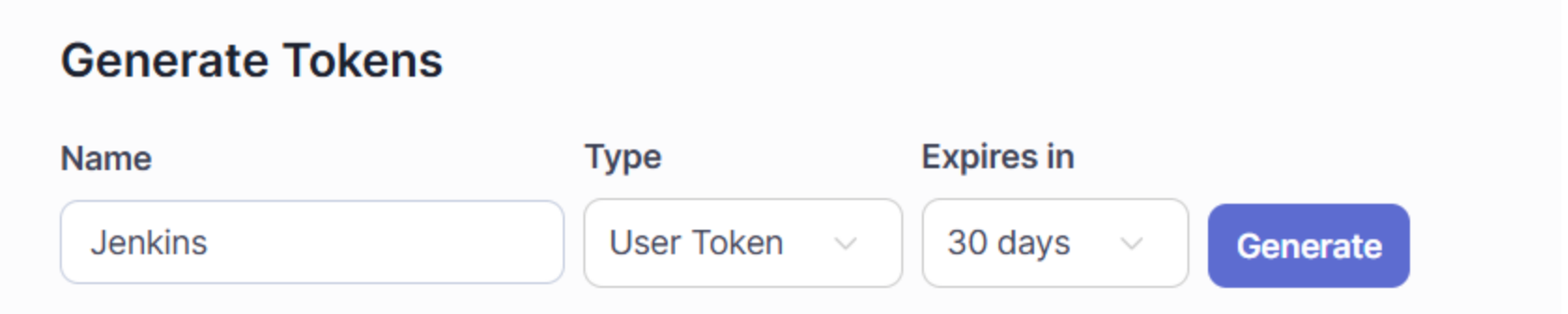
Install SonarQube

- Download and extract
- Set environment variable **SONAR_JAVA_PATH** as C:\Program Files\Java\jdk-21\bin\java.exe
- Run as Admin **StartSonar.bat**
- **Open** http://localhost:9000/
- Login as : admin and password : admin then change the password

Connect SonarQube with Jenkins

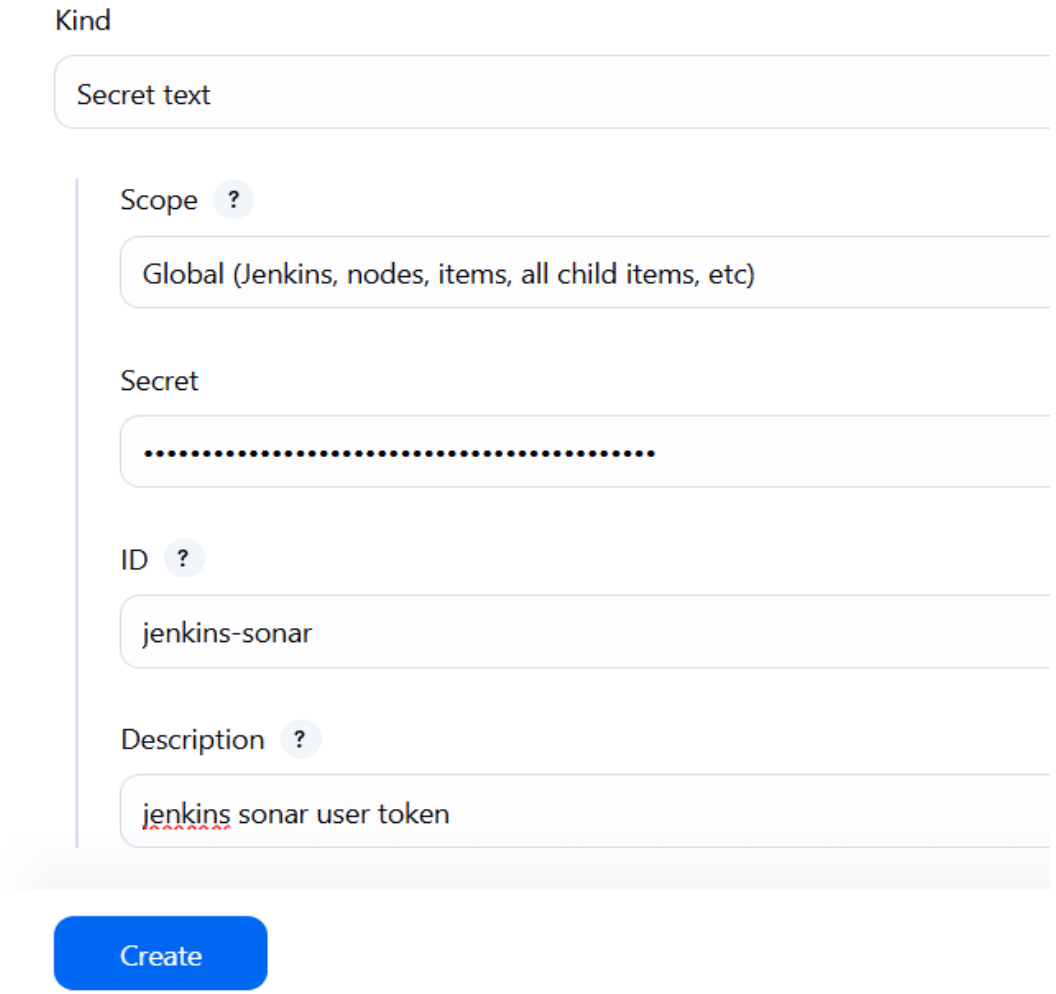
1. Generate SonarQuebe user token

   My Account --> security --> Generate Tokens and generate a token(can give any name as token name)
   Copy the token **squ_ea87de659097d6d935027a914d704d0cbebbde61**



2. Add user token to Jenkins as a credential

   Dashboard --> Manage Jenkins --> Credentials --> Global --> Add Credentials



Secret = copied user token and ID and Description can be anything.

3. Install the below plugins to Jenkins

Dashboard --> Manage Jenkins --> Plugins -- > Available Plugins

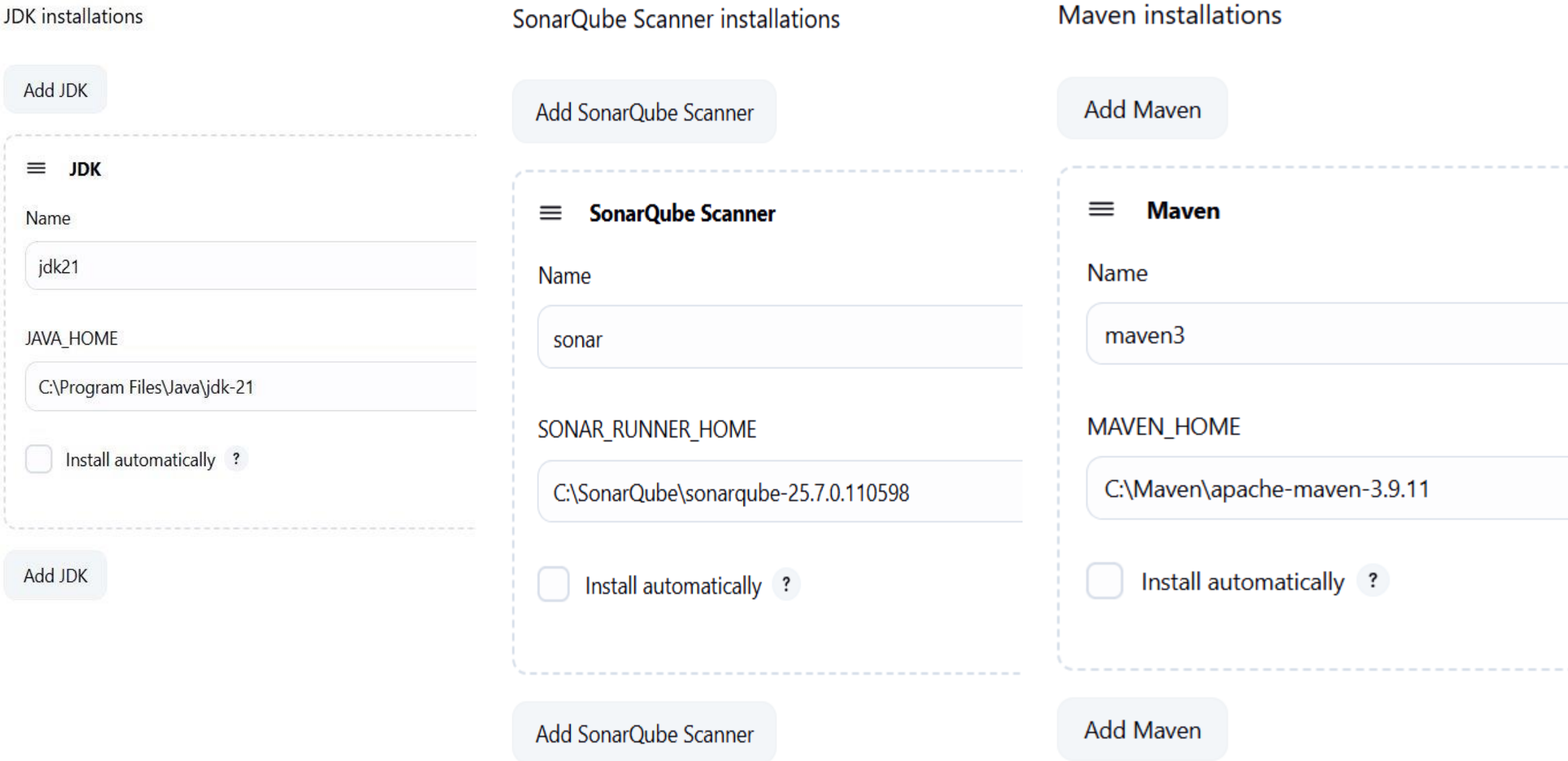
- Pipeline Stage View
- Maven Integration
- SonarQube Scanner

4. Add the below tools to Jenkins

Dashboard --> Manage Jenkins --> Tools

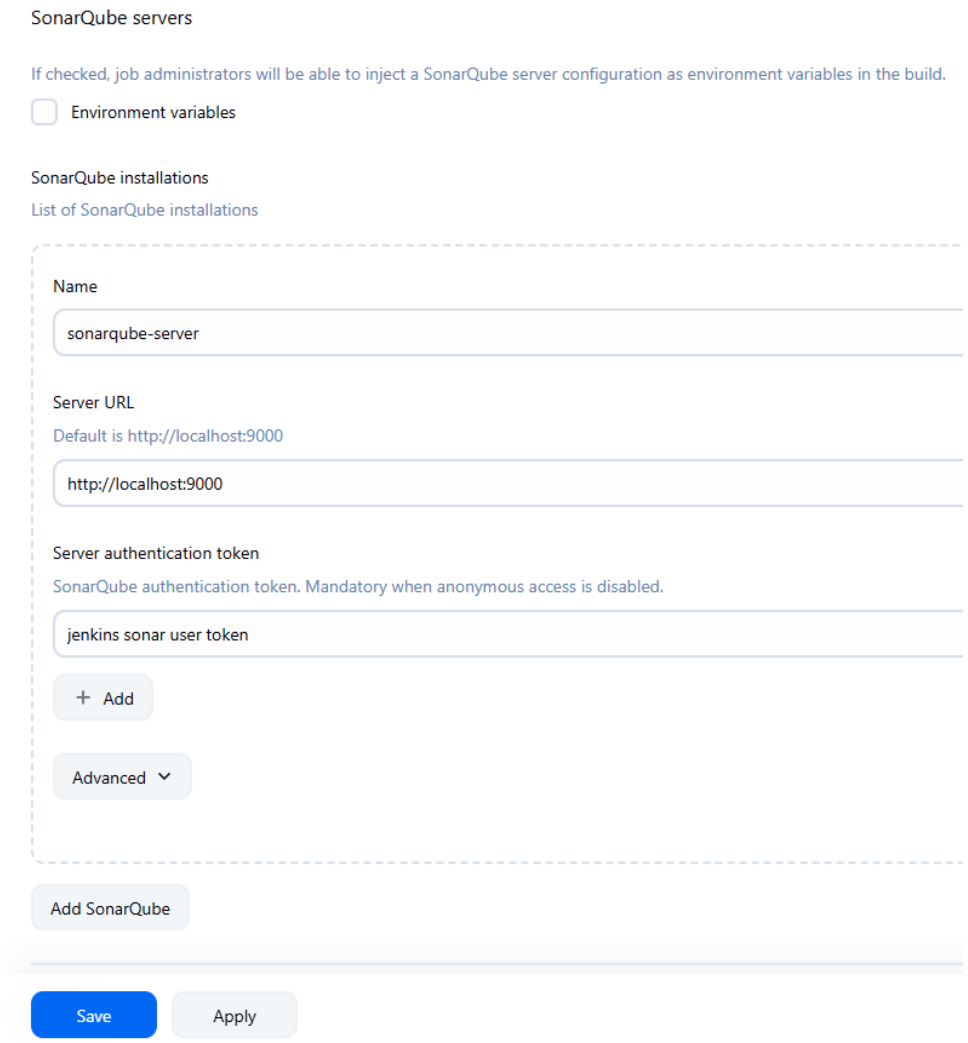
- JDK installations
- SonarQube Scanner installations
- Maven installations

The paths should be set as your computer locations of the applications.
(JAVA_HOME, SONAR_RUNNER_HOME…)


JDK installations
Add JDK
JDK
Name
jdk21
JAVA_HOME
C:\Program Files\Java\jdk-21
Install automatically
Add JDK
SonarQube Scanner installations
Add SonarQube Scanner
SonarQube Scanner
Name
sonar
SONAR_RUNNER_HOME
C:\SonarQube\sonarqube-25.7.0.110598
Install automatically
Add SonarQube Scanner
Maven installations
Add Maven
Maven
Name
maven3
MAVEN_HOME
C:\Maven\apache-maven-3.9.11
Install automatically
Add Maven

5. Add SonarQube server to Jenkins

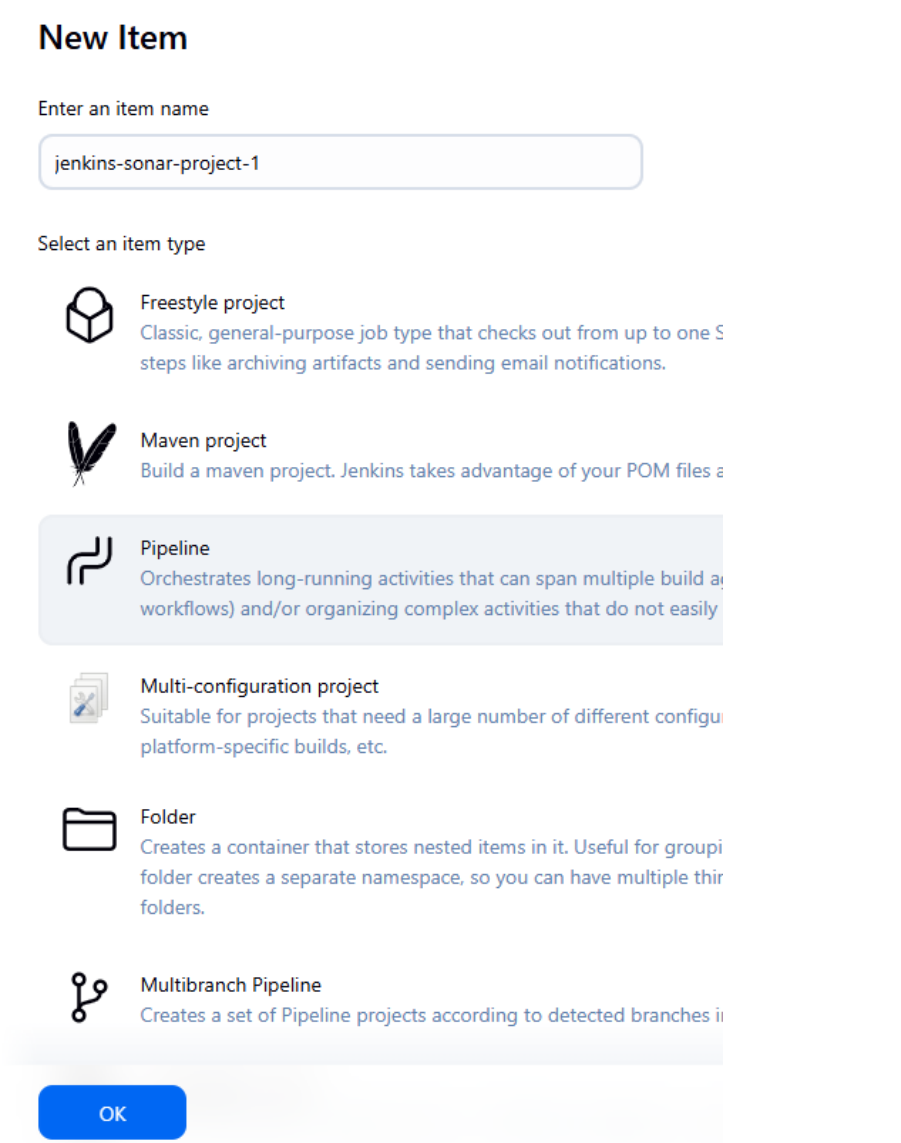
Dashboard --> Manage Jenkins -- > System



- Name can be any name. But use in future.
- Server URL is the SonarQube local URL
- Snanner authentication token is user token (saved at step 2)

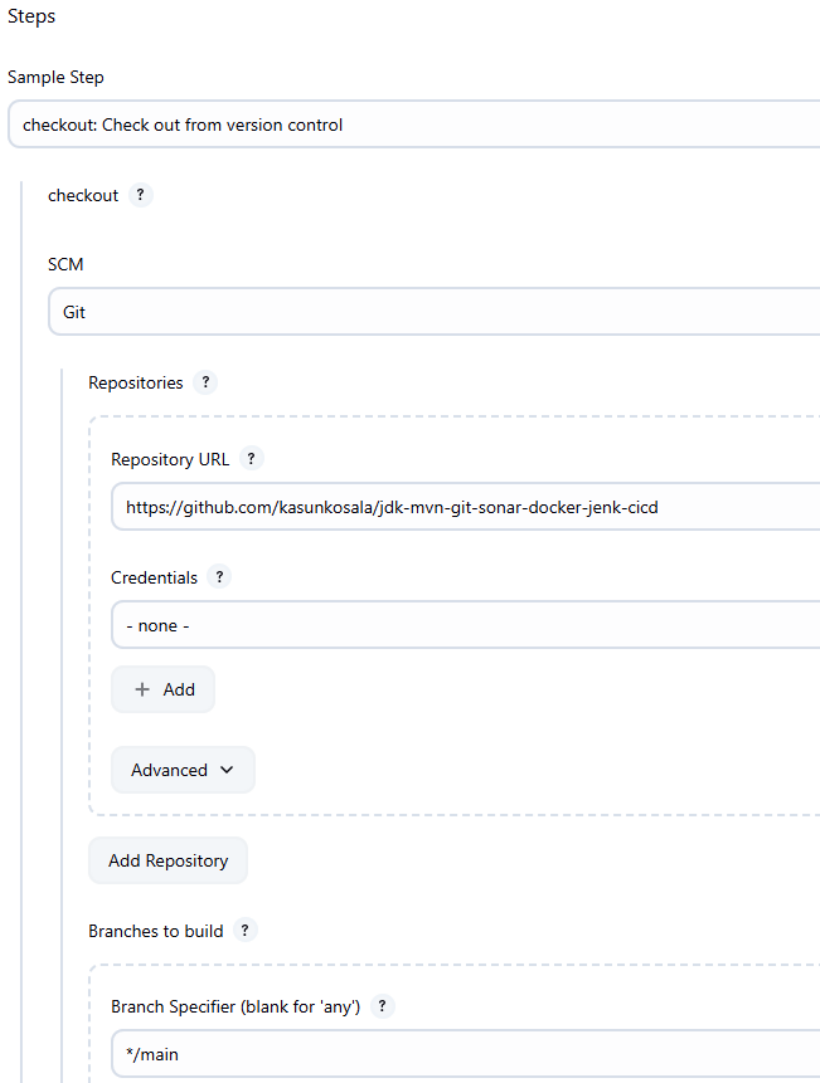
6. Create a pipeline

Dashboard--> New Item



7. generate the git code

Dashboard --> jenkins-sonar-project-1 --> Pipeline Syntax



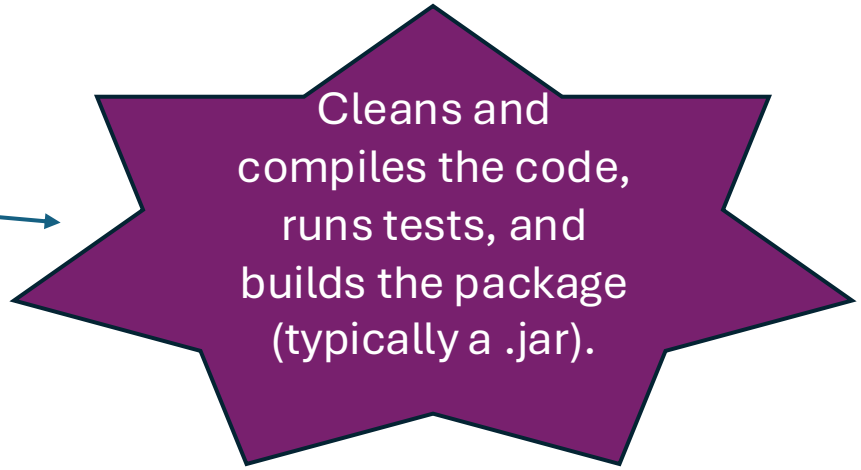
- Repository URL is the our java code containing GitHub URL.
- Change the Branch Spiffier from master to main.
- Then click "**Generate Pipeline Script**" and copy the code.

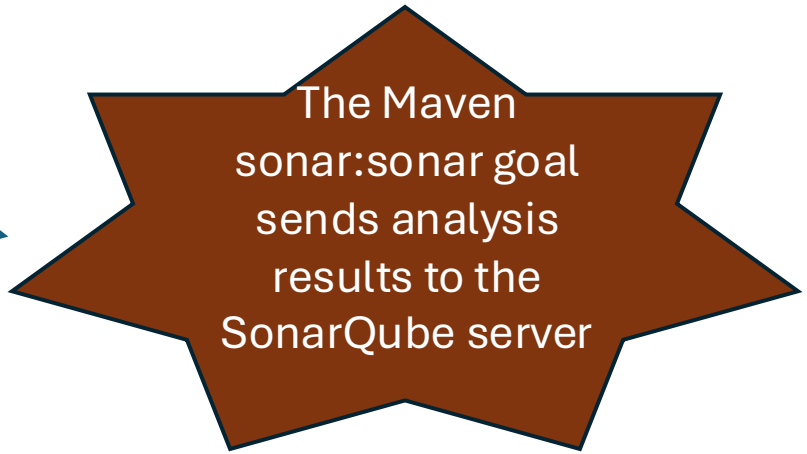
checkout scmGit(branches: [[name: '*/main']], extensions: [], userRemoteConfigs: [[url: 'https://github.com/kasunkosala/jdk-mvn-git-sonar-docker-jenk-cicd']])

8. Generate the final code

```
pipeline {

    agent any

    tools {
        jdk 'jdk21'
        maven 'maven3'
    }

    environment {
        SONARQUBE_SERVER = 'sonarqube-server'
    }

    stages {
        stage('Checkout') {
            steps {
               checkout scmGit(branches: [[name: '*/main']], extensions: [],
userRemoteConfigs: [[url: 'https://github.com/kasunkosala/jdk-mvn-git-sonar-docker-jenk-
cicd']])
            }
        }
```

```
stage('Maven Build') {
    steps {
        echo 'Running Maven build...'
        bat 'mvn clean install'
    }
}

stage('SonarQube Analysis') {
    steps {
        echo 'Running SonarQube Analysis...'
        withSonarQubeEnv("${env.SONARQUBE_SERVER}") {
            bat 'mvn sonar:sonar'
        }
   }
}

stage('Quality Gate') {
    steps {
        echo 'Checking Quality Gate...'
      // This stage is usually used with waitForQualityGate() if SonarQube webhook is configured
     // bat 'mvn verify' could also go here if needed
    }
}
}
```



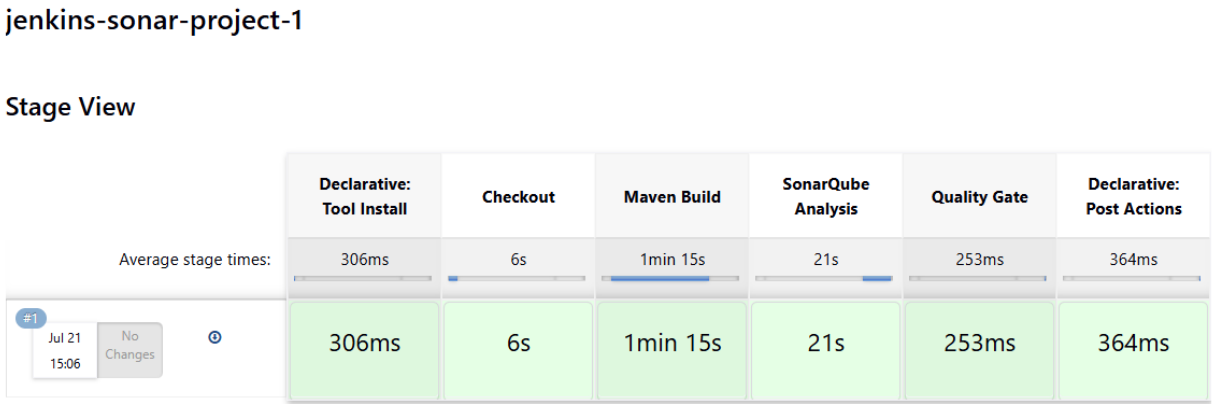


```
post {
    success {
        echo 'Build Successful: Archiving results and publishing tests.'
        junit '**/target/surefire-reports/TEST-*.xml'
        archiveArtifacts artifacts: 'target/*.jar', fingerprint: true
    }

   failure {
        echo 'Build Failed!'
   }
}

}
```

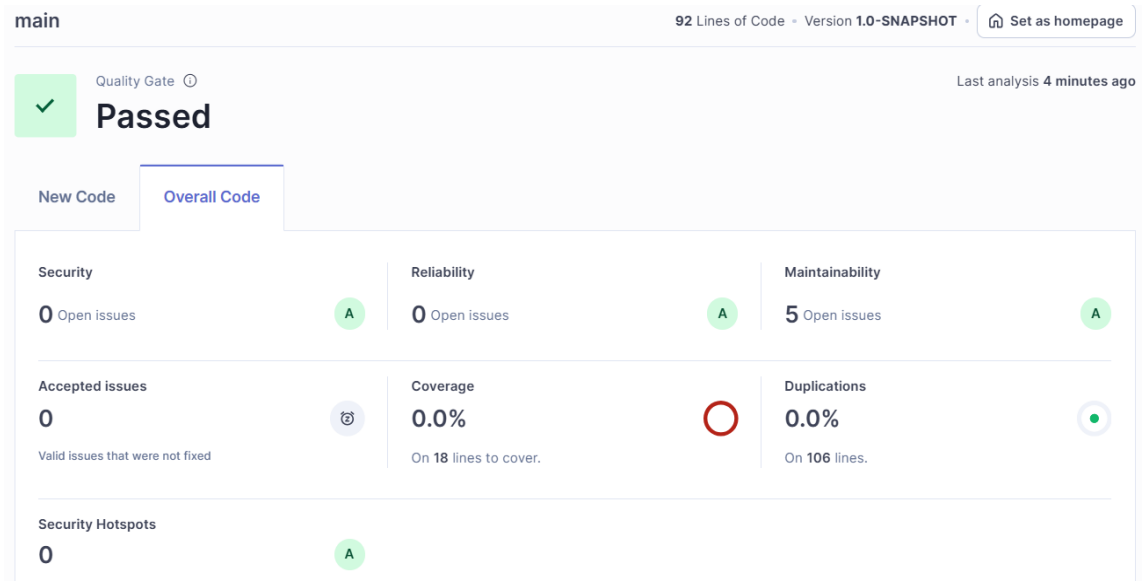
Publishes JUnit test results.
Archives the JAR file built by Maven, making it downloadable from Jenkins UI.

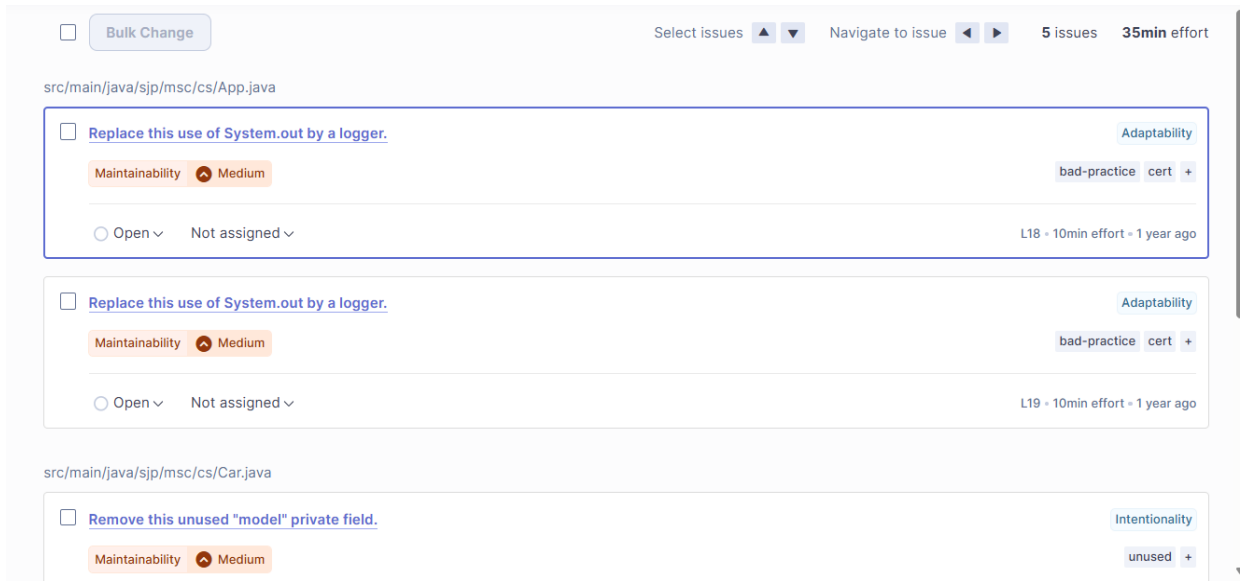
Add the above code as the pipeline script and apply and save.

Then click on Build now...



9. Go to the SonarQube and see the results





End!